OSPF CONTROL-PLANE ANOMALY DETECTION

Section 1 — OSPF Protocol Analysis

OSPF (Open Shortest Path First) is a link-state routing protocol operating at IP protocol number 89. All analysis in this project is conducted exclusively on parsed packet headers — protocol fields, timing, and structure — without any payload inspection.

1.1 OSPF Packet Types

OSPF defines five packet types. Each has distinct timing behavior, multicast characteristics, and fields useful for anomaly detection.

Type 1 — Hello

Role: Neighbor discovery, keepalive, election of DR/BDR on multi-access networks.

Normal timing: Sent every 10 seconds (Hello Interval). If no Hello is received within 40 seconds (Dead Interval = 4x Hello), the neighbor is declared down.

Multicast: Sent to 224.0.0.5 (AllSPFRouters). On multi-access networks, DR and BDR also receive 224.0.0.6.

Header fields visible without payload inspection:

- Network Mask, Hello Interval, Dead Interval, Router Priority
- Router ID, Area ID, Checksum, Authentication Type
- Neighbor field — list of neighbors the sender acknowledges

Anomaly detection value:

- Rate deviation: normal = 0.1 pkt/sec; attack = 10-1000 pkt/sec
- Neighbor field consistency: legitimate Hello lists known neighbors; spoofed Hellos often list none or fake IDs
- Router-ID to source-IP consistency: a given Router-ID should always come from the same IP

Type 2 — Database Description (DBD)

Role: During the Exchange state of adjacency formation, DBD packets advertise the list of LSA headers in the router's LSDB.

Normal timing: Exchanged once during initial adjacency formation. Typically 3-10 packets total between two routers. Not periodic — only occurs at adjacency formation or reset.

Multicast: DBD is unicast between the two routers forming adjacency, not multicast.

Header fields:

- Options, Interface MTU, DBD Sequence Number, I/M/MS flags (Init, More, Master-Slave)

Anomaly detection value:

- Sustained high DBD rate from a single source = DBD Flood attack

- DBD without subsequent LSR within 2-3 seconds = adjacency stuck in ExStart/Exchange
- Repeated DBD with identical sequence number = resend loop anomaly

Type 3 — Link State Request (LSR)

Role: Request specific LSAs from a neighbor during database synchronization.

Normal timing: Follows DBD exchange. A few packets, then stops. Not periodic.

Header fields: LSA Type, Link State ID, Advertising Router (for each requested LSA).

Anomaly value: LSR without prior DBD exchange is structurally anomalous. High LSR rate indicates forced re-synchronization.

Type 4 — Link State Update (LSU)

Role: The primary vehicle for flooding LSA information. Contains one or more LSAs.

Normal timing: Triggered by topology changes. Periodic refresh every 30 minutes (MaxAge prevention). Rate is very low — typically 1-3 LSU packets per topology change, then silence.

Multicast: Sent to 224.0.0.5 (or 224.0.0.6 for DR/BDR). Flooded by all routers that receive it.

LSA Header fields (within LSU, visible without payload):

- LS Age (0-3600 seconds), Options, LS Type (1-5), Link State ID
- Advertising Router ID, LS Sequence Number (0x80000001 to 0x7FFFFFFF), LS Checksum, Length

Anomaly value — this is the richest source of attack signals:

- LS Age = 3600 from unexpected source = MaxAge attack
- Sequence number jumping by more than 1 = Seq++ attack
- Sequence number lower than last known = Replay attack
- High LSU rate = LSA Flood attack
- Same seq+checksum from two different source IPs = Disguised LSA attack

Type 5 — Link State Acknowledgement (LSAck)

Role: Acknowledges received LSUs. Sent after processing each LSU.

Normal timing: Immediately follows LSU. 1:1 ratio with LSU packets in balanced traffic.

Anomaly value: LSAck without corresponding LSU = replay artifact. LSU with no LSAck response = dropped/filtered traffic.

1.2 Key OSPF Fields and Their Classification

Field	Type	Normal Value	Anomalous Value	Detection Method
LS Sequence Number	Deterministic	Increments by 1 every 30min	Jump >1 or rollback	Rule-Based
LS Age	Deterministic	0-3600, grows over time	3600 from non-originator	Rule-Based
Router ID	Deterministic	Stable, maps 1:1 to src IP	Same ID, different src IP	Rule-Based
Area ID	Deterministic	Matches known area for interface	Unknown area from src	Rule-Based
LS Checksum	Deterministic	Valid Fletcher checksum	Duplicate checksum/seq	Rule-Based
Inter-arrival time	Statistical	~10s (Hello), ~1800s (LSU)	0.01-0.1s under attack	ML
Packets per second	Statistical	0.1/s Hello, 0.001/s LSU	>10/s under flood	ML
Packet size	Statistical	48-64B Hello, 100-500B LSU	Uniform under flood	ML
Flow symmetry	Behavioral	Balanced req/resp ratio	Unidirectional flooding	ML
DBD-to-LSR ratio	Behavioral	~1:1 during adjacency	DBD without LSR	Hybrid

Section 3 — Attack Selection and Classification

3.1 Final Attack List

Attack	Detection Method	Label	Primary Signal
Hello Flood	Rule_based	ospf_hello_flood	Inter-arrival collapse, rate spike, type ratio skew
LSA Flood (DoS)	Rule-Based	ospf_lsa_flood	LSU rate, seq increment rate, age uniformity
DBD Flood	Rule-Based	ospf_dbd_flood	DBD rate, missing LSR follow-up
Seq++ / Replay	Rule-Based	ospf_seq_attack	Sequence number jump or rollback per router_id
MaxAge LSA Abuse	Rule-Based	ospf_maxage	LS Age == 3600 from non-originating source
Area ID Manipulation	Rule-Based	ospf_area_manip	Area ID not in known set for that src_ip
Router-ID Spoofing	Rule-Based	ospf_routerid_spoof	Same router_id from different src_ip
Disguised LSA Attack	Rule-Based	ospf_disguised_lsa	Duplicate seq+checksum from different src
Normal	N/A	benign	Baseline OSPF convergence and keepalive

3.2 Attack Mechanisms and Observable Metadata

Attack 1 — Hello Flood

Mechanism: Attacker sends OSPF Hello packets at 10-1000x the normal rate toward 224.0.0.5. Each Hello is structurally valid but originates from the attacker's IP with a fake or real Router-ID. Legitimate routers process each packet, attempt neighbor formation, exhaust CPU and neighbor table memory.

Observable metadata anomalies:

- inter_arrival_time_mean drops from ~10s to 0.01-0.1s
- packets_per_second spikes to 10-1000
- ospf_type_1_ratio approaches 1.0 (100% Hello, no LSU/LSAck)
- unique_router_ids_per_window stays at 1 (attacker repeating)
- flow_symmetry_ratio breaks (no responses proportional to flood)

Why ML: The exact threshold between a legitimate burst and an attack varies by network. Isolation Forest learns the baseline and flags statistical deviation.

Attack 2 — LSA Flood

Mechanism: Attacker sends continuous LSU packets with rapidly incrementing LS Sequence Numbers. Each LSU is accepted, flooded, and triggers SPF recalculation. The LSDB fills with fake topology entries, CPU is exhausted by Dijkstra computation.

Observable metadata anomalies:

- lsa_packets_per_second: 10-1000/s vs normal 0.001/s
- lsa_seq_increment_per_second: attacker increments every 0.1s; normal is once per 30min
- lsa_age_mean: always near 0 (attacker sends fresh LSAs continuously)
- lsa_age_std: near 0 (no age variety — all packets identical in age)
- type_3_5_count_in_window dominates

Attack 3 — DBD Flood

Mechanism: Attacker sends continuous DBD packets toward victim routers, keeping them stuck in ExStart/Exchange state. Normal DBD exchange completes in 3-10 packets within 2-3 seconds. A flood prevents Full state adjacency.

Observable metadata anomalies:

- dbd_rate_per_source exceeds baseline
- DBD not followed by LSR within expected window
- Repeated DBD with same sequence number (resend loop)
- Adjacency never reaches Full state (no LSU/LSAck follows)

Attack 4 — Seq++ / Replay

Mechanism: Seq++ sends LSAs with sequence numbers jumping far ahead (forcing other routers to accept a 'newer' update). Replay sends old LSAs with sequence numbers lower than the current value (to override newer entries with stale data).

Detection rule: Maintain per-router_id sequence number tracking. Any jump > 1 or rollback < current value is deterministically flagged.

Attack 5 — MaxAge LSA Abuse

Mechanism: Attacker sends LSU with LS Age = 3600 (MaxAge). Receiving routers flush the corresponding LSA from their LSDB, removing the associated route. Normally, MaxAge is only sent by the originating router during a graceful withdrawal.

Detection rule: LS Age = 3600 from any source that is not the Advertising Router of that LSA = anomaly.

Attack 6 — Area ID Manipulation

Mechanism: Attacker sends OSPF packets with an Area ID different from the expected area for that network segment. This causes neighbor rejection or topology confusion. Area ID 0.0.0.0 (backbone) from a non-backbone interface is a particularly clear violation.

Detection rule: Maintain known area_id per src_ip. Any OSPF packet with an unexpected Area ID from that source is flagged.

Attack 7 — Router-ID Spoofing

Mechanism: Attacker sends Hello or LSU packets claiming a Router-ID already used by a legitimate router (e.g., R1=1.1.1.1) but from a different source IP. Receiving routers may disrupt adjacency with the legitimate router.

Detection rule: Maintain router_id → src_ip mapping. Any Hello/LSU where the Router-ID maps to a different src_ip than previously observed = spoofing.

Attack 8 — Disguised LSA Attack

Mechanism: Attacker sends a Trigger LSA to R1 (with spoofed Router-ID = R1), causing R1 to send a Fight-Back LSA. Simultaneously, attacker sends a Disguised LSA to R2 with identical seq+checksum+age as R1's upcoming fight-back but with false topology. R2 accepts the disguised LSA and rejects R1's legitimate correction. False topology persists until next LSA refresh (up to 30 minutes).

How attacker knows R1's fields: OSPF headers are unencrypted and multicast to 224.0.0.5. Attacker passively sniffs seq number, checksum, and age from legitimate LSAs before launching.

Detection rule: Two LSAs with identical {seq, checksum, advertising_router} arriving from different source IPs within 1-10 seconds = Disguised LSA attack. Fight-Back LSA rejected by neighbor within 5 seconds of trigger = secondary signal.

Section 4 — Rule-Based Detection Engine

4.1 Complete Rule-Based Detection Script

The following script parses OSPF PCAPs and applies all rule-based detectors. It requires scapy and scapy-contrib for OSPF layer support.

```
#!/usr/bin/env python3
# ospf_rule_detector.py
# Rule-based OSPF anomaly detector — metadata only, no payload

from scapy.all import rdpcap, IP
from scapy.contrib.ospf import OSPF_Hdr, OSPF_Hello, OSPF_LSUpd
from collections import defaultdict
import pandas as pd
import json

KNOWN_AREA_IDS = {'0.0.0.0'}   # update per topology
SEQ_JUMP_THRESHOLD = 5
MAXAGE = 3600

class OSPFRuleEngine:
    def __init__(self):
        self.router_id_to_ip = {}      # router_id -> first seen src_ip
        self.lsa_seq_tracker = {}       # (adv_router, lsa_id) -> last seq
        self.recent_lsas = []           # list of (time, seq, checksum, adv_router,
src_ip)
        self.alerts = []

    def _alert(self, rule, pkt_time, src_ip, details):
        self.alerts.append({
            'time': pkt_time, 'rule': rule,
            'src_ip': src_ip, 'details': details
        })
        print(f'[ALERT] {rule} | {src_ip} | {details}')

    def check_hello(self, pkt, ts):
        if not pkt.haslayer(OSPF_Hello): return
        ospf = pkt[OSPF_Hdr]
        src = pkt[IP].src
        router_id = ospf.src
        area = ospf.area

        # Rule R1: Router-ID Spoofing
        if router_id in self.router_id_to_ip:
            if self.router_id_to_ip[router_id] != src:
                self._alert('ROUTERID_SPOOF', ts, src,
```

```
                    f'Router-ID {router_id} previously from
{self.router_id_to_ip[router_id]}')
        else:
            self.router_id_to_ip[router_id] = src

        # Rule R2: Area ID Manipulation
        if area not in KNOWN_AREA_IDS:
            self._alert('AREA_MANIPULATION', ts, src,
                f'Unknown Area-ID {area}')

    def check_lsu(self, pkt, ts):
        if not pkt.haslayer(OSPF_LSUpd): return
        ospf = pkt[OSPF_Hdr]
        src = pkt[IP].src
        lsas = pkt[OSPF_LSUpd].lsalist

        for lsa in lsas:
            adv = lsa.adrouter if hasattr(lsa,'adrouter') else None
            lsid = lsa.id if hasattr(lsa,'id') else None
            seq  = lsa.seq  if hasattr(lsa,'seq')  else None
            age  = lsa.age  if hasattr(lsa,'age')  else None
            chk  = lsa.chksum if hasattr(lsa,'chksum') else None

            if seq is None: continue
            key = (adv, lsid)

            # Rule R3: Sequence Rollback (Replay Attack)
            if key in self.lsa_seq_tracker:
                last_seq = self.lsa_seq_tracker[key]
                if seq < last_seq:
                    self._alert('SEQ_REPLAY', ts, src,
                        f'LSA {key} seq rolled back: {last_seq} -> {seq}')
                # Rule R4: Sequence Jump (Seq++ Attack)
                elif seq - last_seq > SEQ_JUMP_THRESHOLD:
                    self._alert('SEQ_JUMP', ts, src,
                        f'LSA {key} seq jumped: {last_seq} -> {seq} (delta={seq-
last_seq})')
            self.lsa_seq_tracker[key] = seq

            # Rule R5: MaxAge Abuse
            if age == MAXAGE and adv != src:
                self._alert('MAXAGE_ABUSE', ts, src,
                    f'MaxAge LSA for {adv} from non-originating src {src}')

            # Rule R6: Disguised LSA Detection
            cutoff = ts - 10.0
            self.recent_lsas = [(t,s,c,a,ip) for (t,s,c,a,ip)
                                in self.recent_lsas if t >= cutoff]
```

```
            for (prev_t, prev_seq, prev_chk, prev_adv, prev_ip) in
self.recent_lsas:
                if (prev_seq == seq and prev_chk == chk and
                    prev_adv == adv and prev_ip != src):
                    self._alert('DISGUISED_LSA', ts, src,
                        f'Duplicate seq={seq} chk={chk} adv={adv} from {src} vs
{prev_ip}')
            self.recent_lsas.append((ts, seq, chk, adv, src))

    def process_pcap(self, filepath):
        pkts = rdpcap(filepath)
        for pkt in pkts:
            if not pkt.haslayer(IP): continue
            if pkt[IP].proto != 89: continue
            ts = float(pkt.time)
            self.check_hello(pkt, ts)
            self.check_lsu(pkt, ts)
        return self.alerts

if __name__ == '__main__':
    import sys
    engine = OSPFRuleEngine()
    alerts = engine.process_pcap(sys.argv[1])
    print(f'\nTotal alerts: {len(alerts)}')
    pd.DataFrame(alerts).to_csv('rule_alerts.csv', index=False)
```

4.2 DBD Flood Rate Checker

```
# dbd_flood_check.py — sliding window DBD rate detector
from scapy.all import rdpcap, IP
from scapy.contrib.ospf import OSPF_Hdr, OSPF_DBDesc
from collections import deque

DBD_RATE_THRESHOLD = 5   # DBD packets per 5-second window
WINDOW_SECONDS = 5
LSR_FOLLOWUP_WINDOW = 3  # seconds to wait for LSR after DBD

def detect_dbd_flood(filepath):
    pkts = rdpcap(filepath)
    src_windows = {}   # src_ip -> deque of timestamps
    dbd_times = {}     # src_ip -> last DBD time
    lsr_seen = {}      # src_ip -> last LSR time
    alerts = []

    for pkt in pkts:
        if not pkt.haslayer(IP) or pkt[IP].proto != 89: continue
        ts = float(pkt.time)
        src = pkt[IP].src
```

```
        ospf = pkt[OSPF_Hdr]

        if ospf.type == 2:  # DBD
            q = src_windows.setdefault(src, deque())
            q.append(ts)
            while q and ts - q[0] > WINDOW_SECONDS:
                q.popleft()
            if len(q) > DBD_RATE_THRESHOLD:

alerts.append({'rule':'DBD_FLOOD','src':src,'ts':ts,'count':len(q)})
                print(f'[ALERT] DBD_FLOOD: {src} sent {len(q)} DBDs in
{WINDOW_SECONDS}s')
            dbd_times[src] = ts

        elif ospf.type == 3:  # LSR
            lsr_seen[src] = ts

    # Check for DBDs without LSR followup
    for src, last_dbd in dbd_times.items():
        last_lsr = lsr_seen.get(src, 0)
        if last_lsr < last_dbd - LSR_FOLLOWUP_WINDOW:
            alerts.append({'rule':'DBD_NO_LSR','src':src,
                           'ts':last_dbd,'detail':'No LSR after DBD'})
            print(f'[ALERT] DBD_NO_LSR: {src} sent DBD with no LSR follow-up')
    return alerts
```